

Reviewer Pack — Fourier Coefficient Product Lower Bound on Disjointness Comm...

Entry cd6772561f42 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 12:13:00 UTC

Fourier Coefficient Product Lower Bound on Disjointness Communication Complexity

Entry ID: cd6772561f42 Verdict: INCONCLUSIVE Recorded: 2026-05-08 12:13:00 UTC

1. Conjecture

Field A (mathematical branch): FOURIER_ANALYSIS **Field B** (complexity object): COMMUNICATION_COMPLEXITY

Statement:

For a Boolean matrix M representing the disjointness function on n elements, the product of the minimal and maximal absolute Fourier coefficients, $\tau(M) = \min_k |F(k)| * \max_k |F(k)|$, satisfies $\tau(M) \geq c * n$ for some absolute constant $c > 0$.

Rationale (proposer’s reasoning):

The product of the minimal and maximal Fourier coefficients captures the spread of the function’s energy in the Fourier domain. A large product implies significant variation in the Fourier coefficients, which would require more communication to distinguish the inputs.

Taxonomy category: FOURIER_ANALYTIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 52c88023986ae33b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=-9, elapsed=79.8s

5.1 Generated Python source

```
import random
import math

def is_prime(n):
    if n <= 1:
        return False
    if n <= 3:
        return True
    if n % 2 == 0 or n % 3 == 0:
        return False
    i = 5
    while i * i <= n:
        if n % i == 0 or n % (i + 2) == 0:
            return False
        i += 6
    return True

def generate_primes(n):
    primes = []
    num = 2
    while len(primes) < n:
        if is_prime(num):
            primes.append(num)
        num += 1
    return primes

def gaussian_elimination(A, b):
    n = len(b)
    for i in range(n):
        max_row = max(range(i, n), key=lambda r: abs(A[r][i]))
        A[i], A[max_row] = A[max_row], A[i]
        b[i], b[max_row] = b[max_row], b[i]
        for j in range(i + 1, n):
            factor = A[j][i] / A[i][i]
            for k in range(i, n):
                A[j][k] -= factor * A[i][k]
            b[j] -= factor * b[i]
    x = [0] * n
    for i in range(n - 1, -1, -1):
```

```

        x[i] = (b[i] - sum(A[i][j] * x[j] for j in range(i + 1, n))) / A[i][i]
    return x

def matrix_multiply(A, B):
    m, k = len(A), len(B[0])
    result = [[0] * k for _ in range(m)]
    for i in range(m):
        for j in range(k):
            for l in range(len(B)):
                result[i][j] += A[i][l] * B[l][j]
    return result

def fourier_coefficients(M, n):
    N = 2 ** n
    F = [[0] * N for _ in range(N)]
    for k in range(N):
        for l in range(N):
            sum_val = 0
            for i in range(n):
                sum_val += M[i][l // (2 ** i) % 2] * math.cos(2 * math.pi * k * l / N)
            F[k][l] = sum_val / N
    return F

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
    M = [[random.randint(0, 1) for _ in range(2 ** n)] for _ in range(n)]
    F = fourier_coefficients(M, n)
    min_abs_F = min(abs(F[k][l]) for k in range(len(F)) for l in range(len(F[0])))
    max_abs_F = max(abs(F[k][l]) for k in range(len(F)) for l in range(len(F[0])))
    tau_M = min_abs_F * max_abs_F
    c = 1 / (2 * math.pi)
    conjecture_holds = tau_M >= c * n
    counterexample = "" if conjecture_holds else f"n={n}, tau(M)={tau_M}, expected>=c*n"
    return {
        "metric_name": "tau(M)",
        "metric_value": tau_M,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or generate_primes(30)
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)
    mean_tau_M = sum(r["metric_value"] for r in results) / len(results)
    std_tau_M = math.sqrt(sum((r["metric_value"] - mean_tau_M) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)
    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_tau_M} std={std_tau_M} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_tau_M} std={std_tau_M} support_fraction={support_fraction}")

```

```

else:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

(empty)

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed before producing data, preventing evaluation of support fraction or counterexamples. | next: Re-run test with increased time/memory limits and deterministic seed configuration

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	113397
2	propose	ollama_remote	qwen3:8b	0	0	118826
3	preregistration	ollama_remote	qwen3:8b	0	0	24069
4	novelty	ollama_remote	qwen3:8b	0	0	20666
5	novelty	ollama_remote	qwen3:8b	0	0	13528
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14836
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	42320
8	judge	ollama_remote	qwen3:8b	0	0	130591

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 478232 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/cd6772561f42.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/cd6772561f42.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/cd6772561f42.tar.gz` (if generated)